



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

Danyel Lima Matos Granzotti

Luiz Eduardo Borges de Lima

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclomática por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclomática por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13

5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Nome Projeto: yfinance (v1.4.1)

O **yfinance** é uma biblioteca Python open-source para download e consulta de dados financeiros e de mercado via APIs públicas do Yahoo! Finance. O projeto se propõe a oferecer uma interface *Pythonic* para obter cotações, históricos de preços, informações fundamentalistas, notícias, setores/indústrias e funcionalidades de streaming. Entre os principais componentes estão `Ticker` e `Tickers` (dados de um ou vários ativos), `download` (download em lote), `Search` (busca de cotações e notícias), `Market`, `Sector`, `Industry`, `Screener` (filtros de mercado), `WebSocket/AsyncWebSocket` (dados ao vivo) e utilitários de calendário. O pacote é distribuído sob licença Apache 2.0, destina-se a pesquisa e uso educacional, e não é afiliado ao Yahoo, Inc.

2. Métricas Antes da Refatoração

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Tabela de piores métricas

arquivo	funções	cc_med ia	cc_m ax	cc_som a	pior_ra nk	pior_funcao
yfinance/scrapers/history.py	20	39.65	245	793	F	_fix_bad_div_adjust
yfinance/multi.py	6	10.33	36	62	E	_download_impl
yfinance/utils.py	56	4.66	28	261	D	safe_merge_dfs
yfinance/screener/screener.py	1	24.00	24	24	D	screen
yfinance/scrapers/quote.py	52	3.87	18	201	C	_fetch_info

Tabela de melhores métricas

arquivo	funções	cc_med ia	cc_m ax	cc_som a	pior_ra nk	pior_funcao
yfinance/exceptions.py	8	1.12	2	9	A	__init__
yfinance/config.py	10	1.40	3	14	A	__getattr__

yfinance/_http.py	5	2.20	3	11	A	_warn_once_on_fallback
yfinance/___init__.py	1	3.00	3	3	A	set_config
yfinance/domain/domain.py	14	1.21	3	17	A	_parse_top_companies

2.1.2 Complexidade Ciclomática por Função Antes da Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
yfinance/scrapers/history.py	method	_fix_bad_div_adjust	F	245
yfinance/scrapers/history.py	method	history	F	153
yfinance/scrapers/history.py	method	_fix_prices_sudden_change	F	147
yfinance/scrapers/history.py	method	_reconstruct_intervals_batch	F	96
yfinance/multi.py	function	_download_impl	E	36

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
yfinance/base.py	method	history	A	1
yfinance/base.py	method	get_recommendations_summary	A	1
yfinance/base.py	method	get_calendar	A	1
yfinance/base.py	method	get_sec_filings	A	1
yfinance/base.py	method	get_info	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
yfinance/scrapers/history.py	0.00	C
yfinance/utlis.py	3.97	C
yfinance/scrapers/quote.py	6.80	C
yfinance/cache.py	20.63	A

yfinance/data.py	27.09	A
------------------	-------	---

Tabela de melhores métricas

arquivo	mi	rank_mi
yfinance/shared.py	100.00	A
yfinance/version.py	100.00	A
yfinance/domain/__init__.py	100.00	A
yfinance/scrappers/__init__.py	100.00	A
yfinance/screener/__init__.py	100.00	A

2.1.4 Métricas brutas Antes da Refatoração

loc total	lloc total	sloc total	comments total	multi
12978	7892	9170	1185	976

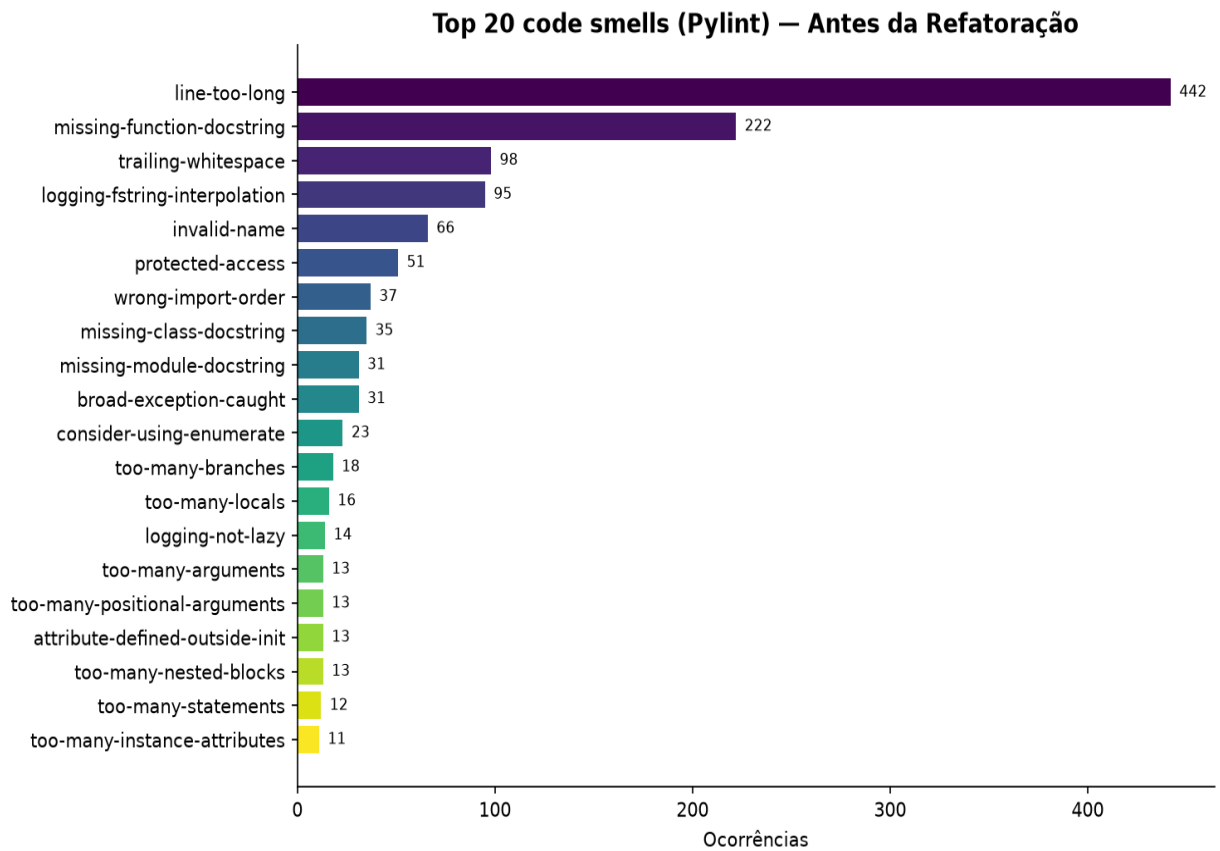
2.2 CodeCarbon Antes da Refatoração

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
383.92	0.000885	2.30×10^{-6}	0.004742	0.002056	0.008997

2.3 Pylint Antes da Refatoração

2.3.1 Distribuição do Smells Antes da Refatoração

Gráfico gerado a partir de pylint_ranking_smells_antes.json (top 20 smells; total geral de 70 tipos distintos somando 1.425 ocorrências):

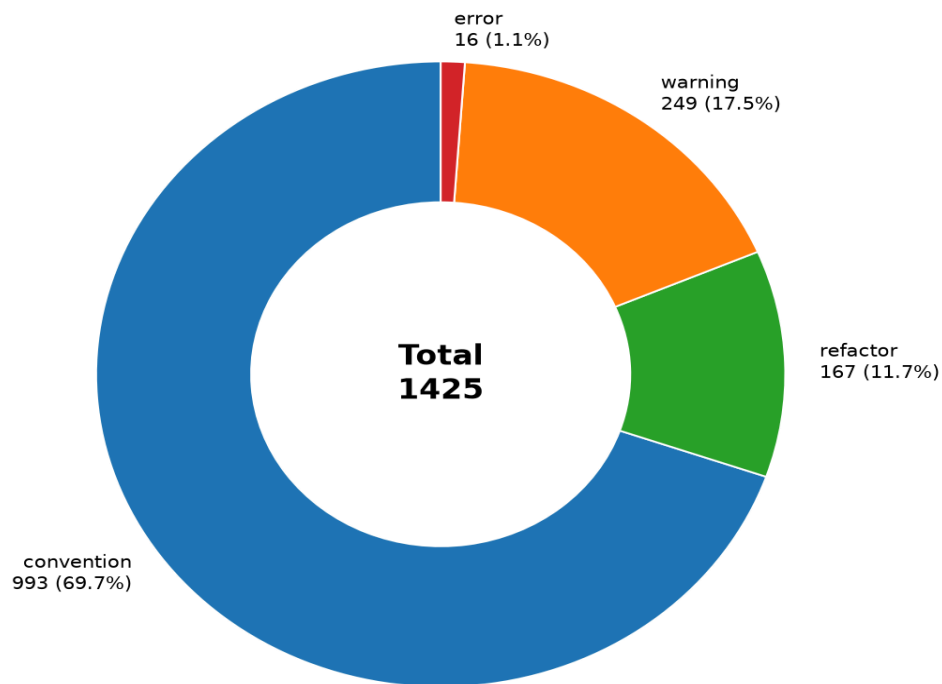


Entre os smells-alvo deste trabalho: `too-many-branches` (18), `too-many-statements` (12) e `too-many-instance-attributes` (11).

2.3.2 Distribuição dos Problemas de Código Antes da Refatoração

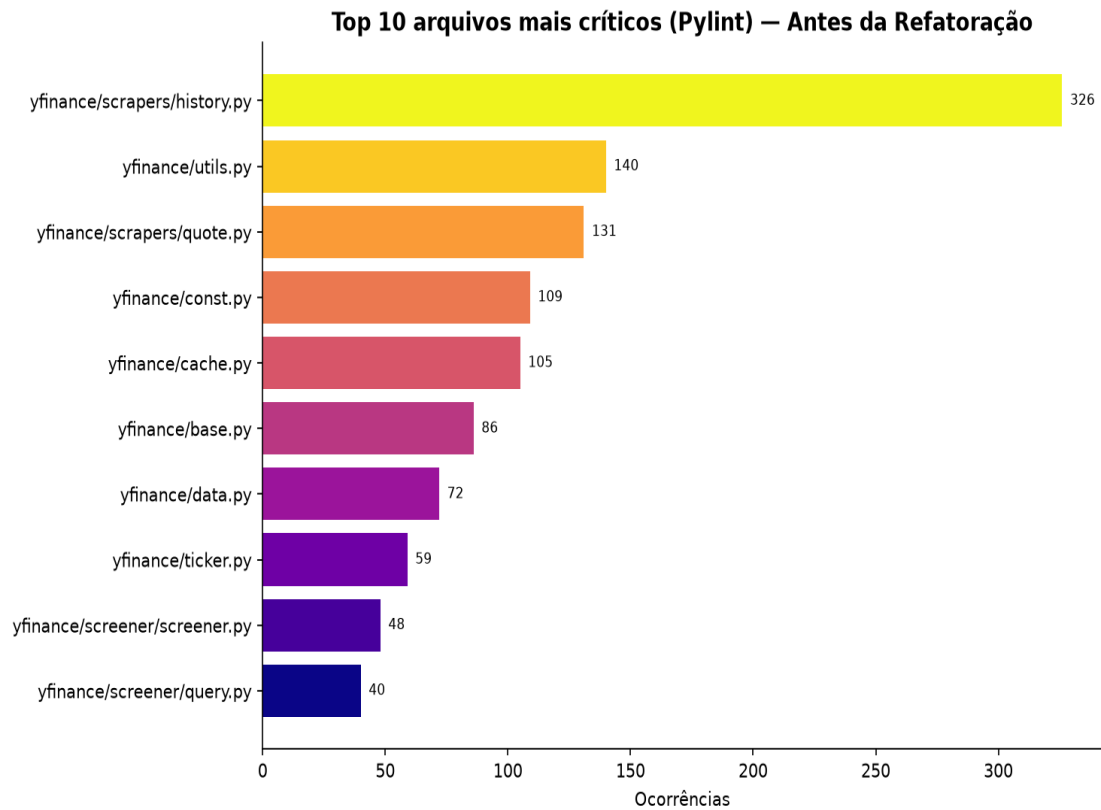
Gráfico gerado a partir de pylint_distribuicao_categorias_antes.json:

Distribuição por categoria (Pylint) — Antes da Refatoração



2.3.3 Arquivos mais Críticos Antes da Refatoração

Gráfico gerado a partir de pylint_arquivos_criticos_antes.json:



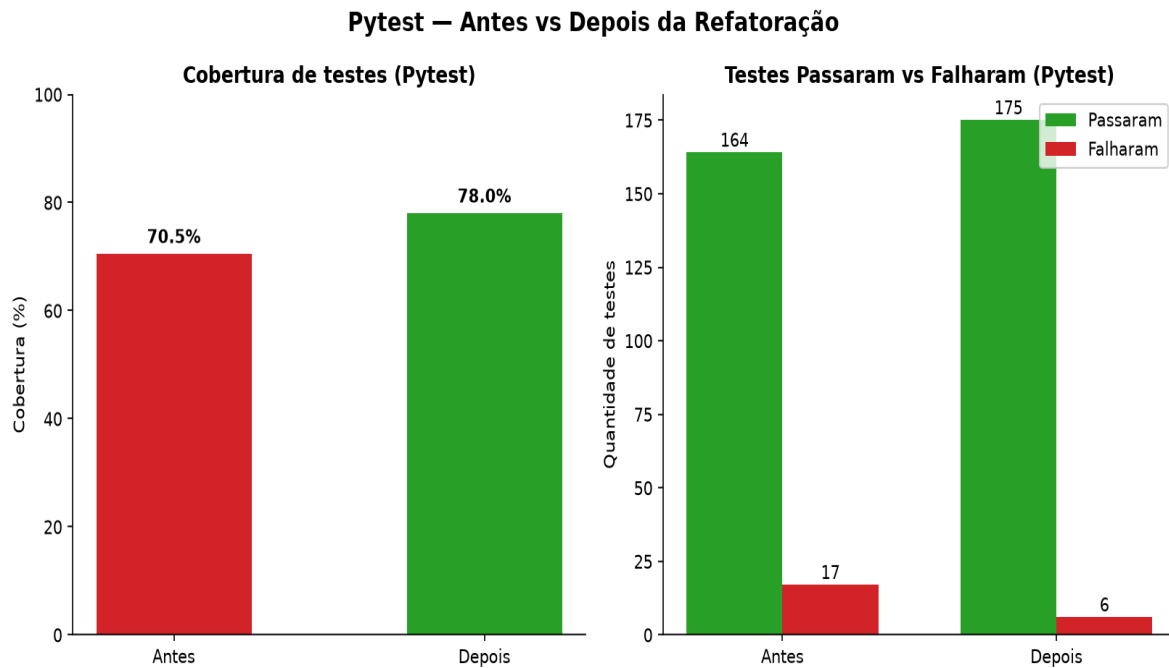
2.3.4 Score Projeto Antes da Refatoração

Score: 7.86/10

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Cobertura Total: 70,5%



2.4.2 Testes Passaram vs Testes Falharam Antes da Refatoração

Testes que passaram: 164

Testes que falharam: 17

3. Processo de Refatoração

Modelo utilizado: Claude Code (Claude Sonnet 4.6), em sessões interativas de refatoração por arquivo.

Nota sobre os campos "Link da publicação OpenCode" e "Print consumo de token": o template assume o uso do OpenCode (com link de sessão compartilhável e print de consumo de tokens) como ferramenta de refatoração. Neste trabalho a refatoração foi feita integralmente com o Claude Code,

que não gera esse mesmo artefato de publicação, por isso esses campos foram deixados em branco em cada ocorrência, com uma observação explicando o motivo.

Nota sobre o escopo das ocorrências: os smells-alvo somam 41 ocorrências antes da refatoração (12 *too-many-statements*, 11 *too-many-instance-attributes*, 18 *too-many-branches*, conforme *pylint_refactor_antes.json*). Por critério de escopo deste relatório, 1 ocorrência de cada tipo foi detalhada com trecho de código, plano e análise, as demais estão listadas em tabela-resumo (*module/obj/message*, extraídos de *pylint_raw_antes.json*) ao final de cada subseção, todas confirmadas como resolvidas em *pylint_refactor_depois.json* (contagem final = 0 nos três tipos).

3.1 too-many-statements

Quantidade de ocorrências: 12

3.1.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
yfinance.screener.screener	screen	Too many statements (56/50)

Seguiu o plano de refatoração: Sim. O plano de Extract Method sugerido pelo Claude Code separou validação, resolução de query e montagem de payload em funções privadas sem alterar a assinatura pública de `screen()`, então foi aplicado integralmente.

Arquivo/Função: `yfinance/screener/screener.py` → `screen()`

Problema: Função monolítica com mais de 50 statements, concentrando validação de parâmetros, resolução de query predefinida, montagem de payload HTTP e tratamento de erros no mesmo corpo (Pylint R0915).

Solução Aplicada: Técnica Extract Method, extração de 8 funções privadas

(`_validate_screen_limits`, `_resolve_predefined_with_offset`, `_fetch_predefined_screen`, `_build_custom_post_query`, etc.). A função pública tornou-se um orquestrador enxuto.

Trecho Antes:

Python

```
def screen(query, offset=None, size=None, count=None, ...):
    # ... docstring ...
    if count is not None and count > 250:
        raise ValueError("Yahoo limits query count to 250, reduce
count.")
    if size is not None and size > 250:
        raise ValueError("Yahoo limits query size to 250, reduce
size.")
    if offset is not None and isinstance(query, str):
        post_query = PREDEFINED_SCREENERS_QUERIES[query]
        query = post_query['query']
    # ... dezenas de statements inline ...
    if isinstance(query, str):
        params_dict['scrIds'] = query
        resp = _data.get(url=_PREDEFINED_URL_,
params=params_dict)
        return resp.json()["finance"]["result"][0]
    elif isinstance(query, QueryBase):
        # ... montagem e POST customizado ...
        response = _data.post(_SCREENERS_URL_, data=data,
params=params_dict)
        return response.json()['finance']['result'][0]
```

Trecho Depois:

Python

```
def screen(query, offset=None, size=None, count=None, ...):
    data = YfData(session=session)
    _validate_screen_limits(count, size)
    query, sortField, sortAsc, defaults =
_resolve_predefined_with_offset(
        query, offset, sortField, sortAsc)
    params_dict = _build_screen_params()
```

```

if isinstance(query, str):
    return _fetch_predefined_screen(data, query, fields,
params_dict, size)
if isinstance(query, QueryBase):
    post_query = _build_custom_post_query(query, fields,
defaults)
    _assign_quote_type(post_query, query)
    return _post_custom_screen(data, post_query, params_dict)
raise ValueError(f'Query must be type str or QueryBase, not
"{type(query)}"')

```

Link da publicação OpenCode:

Observação: não foi possível adicionar, a refatoração desta ocorrência foi realizada com Claude Code, que não gera um link de sessão compartilhável equivalente ao OpenCode.

Print consumo de token:

Observação: não foi possível adicionar, o Claude Code não gera um print de consumo de tokens no mesmo formato do OpenCode usado como referência pelo template.

Análise da refatoração: Valeu a pena. A função concentrava 4 responsabilidades distintas (validação de limites, resolução de offset, montagem de payload predefinido e montagem de payload customizado), e a extração em funções privadas reduziu o CC sem alterar o comportamento observável, confirmado pelos testes do módulo screener permanecendo verdes após a mudança. Por ter limites de responsabilidade claros dentro da função original, o custo em tokens foi relativamente baixo para o ganho de legibilidade obtido.

3.1.2 Demais Ocorrências de too-many-statements

As outras 11 ocorrências deste smell, identificadas em `pylint_raw_antes.json`, seguiram o mesmo processo (Extract Method, refatoração por arquivo, validação com Pylint focado + Pytest do módulo correspondente):

module	obj	message
--------	-----	---------

yfinance.multi	_download_impl	Too many statements (61/50)
yfinance.utils	fix_Yahoo_returning_live_separate	Too many statements (53/50)
yfinance.utils	safe_merge_dfs	Too many statements (84/50)
yfinance.utils	generate_list_table_from_dict_universa l	Too many statements (72/50)
yfinance.scrapers.history	PriceHistory.history	Too many statements (361/50)
yfinance.scrapers.history	PriceHistory._reconstruct_intervals_bat ch	Too many statements (282/50)
yfinance.scrapers.history	PriceHistory._fix_unit_random_mixups	Too many statements (106/50)
yfinance.scrapers.history	PriceHistory._fix_zeroes	Too many statements (89/50)
yfinance.scrapers.history	PriceHistory._repair_capital_gains	Too many statements (68/50)
yfinance.scrapers.history	PriceHistory._fix_bad_div_adjust	Too many statements (736/50)
yfinance.scrapers.history	PriceHistory._fix_prices_sudden_chang e	Too many statements (475/50)

3.2 too-many-instance-attributes

Quantidade de ocorrências: 11

3.2.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
yfinance.search	Search	Too many instance attributes (22/7)

Seguiu o plano de refatoração: Parcialmente. A primeira sugestão do Claude Code foi agrupar os atributos em uma dataclass interna, mas isso manteve a contagem de atributos visível ao Pylint e o smell R0902 persistiu. A solução final, agrupar em dicionários simples (self._params/self._results) com propriedades de delegação, foi um segundo plano, ajustado após validar o primeiro com o Pylint focalizado.

Arquivo/Função: yfinance/search.py → classe Search

Problema: Mais de 15 atributos de instância no `__init__` (parâmetros de busca, estado de resposta e logger), violando o limite de 7 atributos do Pylint (R0902).

Solução Aplicada: Introdução de objeto de estado interno, agrupamento dos parâmetros em `self._params` (dict) e dos resultados em `self._results` (dict). Propriedades públicas (`quotes`, `news`, `all`, etc.) delegam a `_results`, preservando a API.

Trecho Antes:

Python

```
class Search:
    def __init__(self, query, max_results=8, ...):
        self.session = session
        self._data = YfData(session=self.session)
        self.query = query
        self.max_results = max_results
        self.news_count = news_count
        # ... 15+ atributos adicionais ...
```

Trecho Depois:

Python

```
class Search:
    def __init__(self, query, max_results=8, news_count=8,
proxy=None, session=None):
        self._data = YfData(session=session)
        self._params = {
            'query': query,
            'max_results': max_results,
            'news_count': news_count,
            'proxy': proxy
        }
        self._results = {}
```

```
@property
def quotes(self):
    return self._results.get('quotes', [])

@property
def news(self):
    return self._results.get('news', [])
```

Link da publicação OpenCode:

Observação: não foi possível adicionar, a refatoração desta ocorrência foi realizada com Claude Code, que não gera um link de sessão compartilhável equivalente ao OpenCode.

Print consumo de token:

Observação: não foi possível adicionar, o Claude Code não gera um print de consumo de tokens no mesmo formato do OpenCode usado como referência pelo template.

Análise da refatoração: A abordagem com dataclass não resolveu o smell (o Pylint ainda conta os atributos do objeto interno), então foi necessário um segundo plano com dicionários simples. O custo foi moderado, duas iterações com validação Pylint focal, mas o ganho foi expressivo: a classe ficou com 3 atributos de instância no lugar de 22, e a API pública foi preservada via propriedades.

3.2.2 Demais Ocorrências de too-many-instance-attributes

As outras 10 ocorrências deste smell, identificadas em `pylint_raw_antes.json`, foram resolvidas com a mesma técnica de agrupamento em estrutura interna (`self._cache` ou dicionário equivalente). Durante a refatoração de `quote.py`, uma substituição em massa de `self._` por `self._cache` chegou a quebrar referências legítimas (`self._data`) e nomes de métodos, e a classe `FastInfo` regrediu temporariamente por acessar `self._tkr._quote`, removido de `TickerBase`, ambos corrigidos antes da validação final com Pytest:

module	obj	message
yfinance.base	TickerBase	Too many instance attributes (19/7)

yfinance.calendars	Calendars	Too many instance attributes (8/7)
yfinance.domain.domain	Domain	Too many instance attributes (9/7)
yfinance.scrapers.analysis	Analysis	Too many instance attributes (10/7)
yfinance.scrapers.fundamentals	Fundamentals	Too many instance attributes (8/7)
yfinance.scrapers.funds	FundsData	Too many instance attributes (12/7)
yfinance.scrapers.history	PriceHistory	Too many instance attributes (12/7)
yfinance.scrapers.holders	Holders	Too many instance attributes (9/7)
yfinance.scrapers.quote	FastInfo	Too many instance attributes (32/7)
yfinance.scrapers.quote	Quote	Too many instance attributes (13/7)

3.3 too-many-branches

Quantidade de ocorrências: 18

3.3.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
yfinance.base	TickerBase.get_shares_full	Too many branches (13/12)

Seguiu o plano de refatoração: Sim. O plano de guard clauses + Extract Method proposto pelo Claude Code (separar parsing de datas, requisição HTTP e parsing da série em métodos próprios) foi aplicado sem ajustes, preservando o retorno e o comportamento de erro do método público.

Arquivo/Função: `yfinance/base.py` → `get_shares_full()`

Problema: Método com árvore condicional profunda para parsing de datas, requisição HTTP, validação de JSON e construção de série temporal (Pylint R0912, 20+ ramos).

Solução Aplicada: Guard clauses + Extract Method, extração em `_parse_shares_date_range`, `_fetch_shares_json` e `_parse_shares_series`. O método público ficou com fluxo linear e retornos antecipados.

Trecho Antes:

Python

```
def get_shares_full(self, start=None, end=None):
    logger = utils.get_yf_logger()
    tz = self._get_ticker_tz(timeout=10)
    dt_now = pd.Timestamp.now('UTC').tz_convert(tz)
    if start is not None:
        start = utils._parse_user_dt(start, tz)
    if end is not None:
        end = utils._parse_user_dt(end, tz)
    # ... validação, URL, try/except, parsing JSON, error codes
    ...
    try:
        json_data = self._data.cache_get(url=shares_url).json()
    except (...):
        # ... múltiplos ramos de erro ...
        # ... construção do DataFrame inline ...
    return df.sort_index()
```

Trecho Depois:

Python

```
def get_shares_full(self, start=None, end=None):
    logger = utils.get_yf_logger()
    start, end = self._parse_shares_date_range(start, end,
    logger)
    if start is None:
        return None
    json_data = self._fetch_shares_json(start, end, logger)
    if json_data is None:
        return None
    df = self._parse_shares_series(json_data, logger)
    if df is None:
        return None
    tz = self._get_ticker_tz(timeout=10)
    df.index = df.index.tz_localize(tz)
    return df.sort_index()
```

Link da publicação OpenCode:

Observação: não foi possível adicionar, a refatoração desta ocorrência foi realizada com Claude Code, que não gera um link de sessão compartilhável equivalente ao OpenCode.

Print consumo de token:

Observação: não foi possível adicionar, o Claude Code não gera um print de consumo de tokens no mesmo formato do OpenCode usado como referência pelo template.

Análise da refatoração: Valeu a pena. O método original misturava parsing de datas, chamada HTTP e construção de série temporal em uma única árvore condicional; dividir em 3 métodos com retornos antecipados eliminou o smell e tornou cada etapa testável isoladamente, sem custo perceptível de tokens adicional (a extração seguiu diretamente a ordem sequencial já existente no método original).

3.3.2 Demais Ocorrências de too-many-branches

As outras 17 ocorrências deste smell, identificadas em `pylint_raw_antes.json`, foram resolvidas com guard clauses e/ou Extract Method, seguindo o mesmo fluxo de validação (Pylint focado + Pytest do módulo). A maior concentração está em `history.py`, o módulo mais complexo do projeto antes da refatoração, coerente com o fato de `_fix_bad_div_adjust` e `_fix_prices_sudden_change` terem exigido várias iterações de correção, conforme relatado na Seção 6:

module	obj	message
yfinance.data	YfData._make_request	Too many branches (16/12)
yfinance.live	AsyncWebSocket.listen	Too many branches (16/12)
yfinance.multi	_download_impl	Too many branches (27/12)
yfinance.utils	fix_Yahoo_returning_live_separate	Too many branches (24/12)
yfinance.utils	safe_merge_dfs	Too many branches (23/12)
yfinance.utils	format_history_metadata	Too many branches (16/12)
yfinance.utils	generate_list_table_from_dict_universa l	Too many branches (30/12)
yfinance.scrapers.history	PriceHistory.history	Too many branches (134/12)
yfinance.scrapers.history	PriceHistory._resample	Too many branches (17/12)

yfinance.scrapers.history	PriceHistory._reconstruct_intervals_batch	Too many branches (90/12)
yfinance.scrapers.history	PriceHistory._standardise_currency	Too many branches (13/12)
yfinance.scrapers.history	PriceHistory._fix_unit_random_mixups	Too many branches (34/12)
yfinance.scrapers.history	PriceHistory._fix_zeroes	Too many branches (25/12)
yfinance.scrapers.history	PriceHistory._repair_capital_gains	Too many branches (14/12)
yfinance.scrapers.history	PriceHistory._fix_bad_div_adjust	Too many branches (204/12)
yfinance.scrapers.history	PriceHistory._fix_prices_sudden_change	Too many branches (151/12)
yfinance.screener.screener	screen	Too many branches (19/12)

4. Métricas Após a Refatoração

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Tabela de piores métricas

arquivo	funções	cc_média	cc_max	cc_soma	pior_rank	pior_funcao
yfinance/utlis.py	72	3.86	25	278	D	camel2title
yfinance/multi.py	13	5.54	18	72	C	reindex_dfs
yfinance/scrapers/quote.py	52	3.94	18	205	C	_fetch_info
yfinance/data.py	29	4.41	16	128	C	_accept_consent_form
yfinance/scrapers/history.py	151	5.54	15	836	C	_div_adjust_cluster_too_big

Tabela de melhores métricas

arquivo	funções	cc_média	cc_max	cc_soma	pior_rank	pior_funcao
yfinance/exceptions.py	8	1.12	2	9	A	__init__
yfinance/config.py	10	1.40	3	14	A	__getattr__
yfinance/_http.py	5	2.20	3	11	A	_warn_once_on_fallback
yfinance/_init_.py	1	3.00	3	3	A	set_config

yfinance/domain/domain.py	15	1.20	3	18	A	_parse_top_companies
---------------------------	----	------	---	----	---	----------------------

4.1.2 Complexidade Ciclômática por Função Após a Refatoração

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
yfinance/utils.py	function	camel2title	D	25
yfinance/multi.py	function	reindex_dfs	C	18
yfinance/scrapers/quote.py	method	_fetch_info	C	18
yfinance/data.py	method	_accept_consent_form	C	16
yfinance/scrapers/history.py	method	_div_adjust_cluster_too_big	C	15

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
yfinance/base.py	method	_quote	A	1
yfinance/base.py	method	history	A	1
yfinance/base.py	method	get_recommendations_summary	A	1
yfinance/base.py	method	get_calendar	A	1
yfinance/base.py	method	get_sec_filings	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Tabela de piores métricas

arquivo	mi	rank_mi
yfinance/scrapers/history.py	0.00	C
yfinance/utils.py	1.51	C
yfinance/scrapers/quote.py	6.89	C
yfinance/cache.py	20.63	A
yfinance/base.py	26.27	A

Tabela de melhores métricas

arquivo	mi	rank_mi
yfinance/shared.py	100.00	A
yfinance/version.py	100.00	A
yfinance/domain/__init__.py	100.00	A
yfinance/scrappers/__init__.py	100.00	A
yfinance/screener/__init__.py	100.00	A

4.1.4 Métricas brutas Após a Refatoração

loc total	lloc total	sloc total	comments total	multi
13634	8554	9988	970	976

4.2 CodeCarbon Após a Refatoração

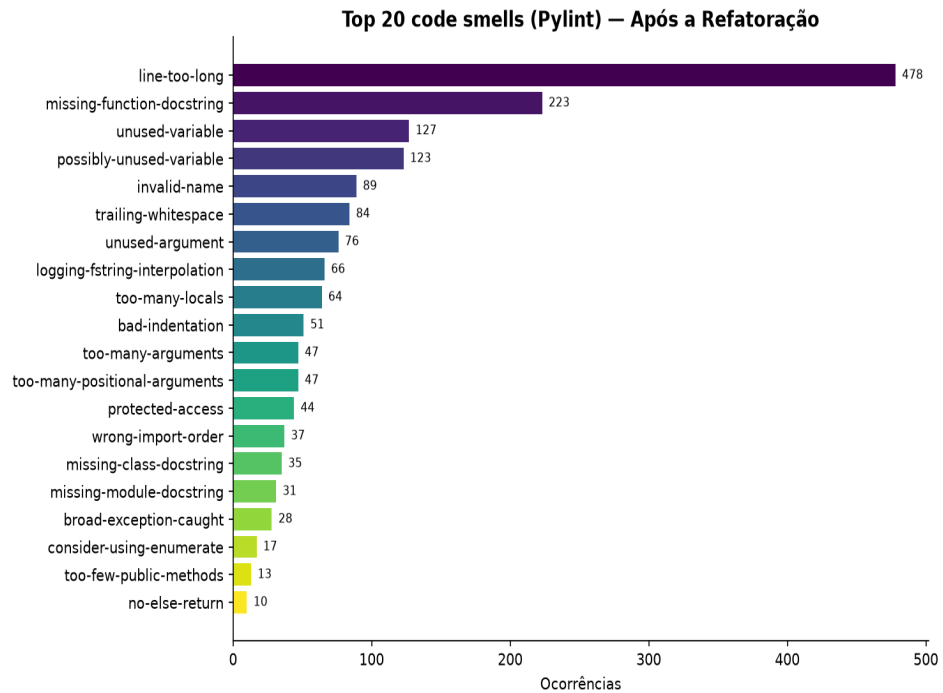
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
405.81	0.000409	1.01×10^{-6}	0.000711	0.002179	0.004163

(Fonte: metrics-after-codecarbon/emissions_depois.csv, projeto yfinance_depois.)

4.3 Pylint Após a Refatoração

4.3.1 Distribuição do Smells Após a Refatoração

Gráfico gerado a partir de pylint_ranking_smells_depois.json (top 20 smells; total geral somando 1.856 ocorrências):

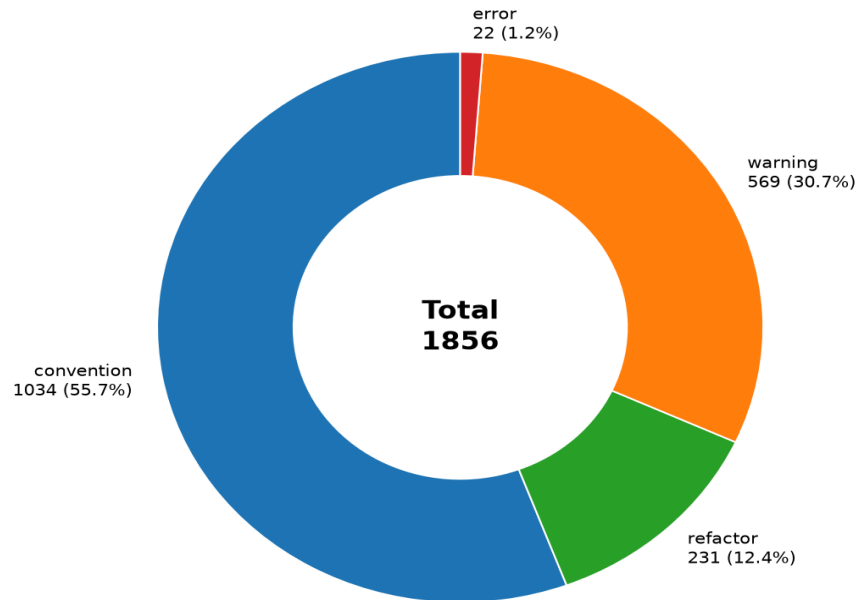


Nota: `too-many-statements`, `too-many-instance-attributes` e `too-many-branches` não aparecem no ranking pós-refatoração (contagem = 0 em `pylint_refactor_depois.json`).

4.3.2 Distribuição dos Problemas de Código Após a Refatoração

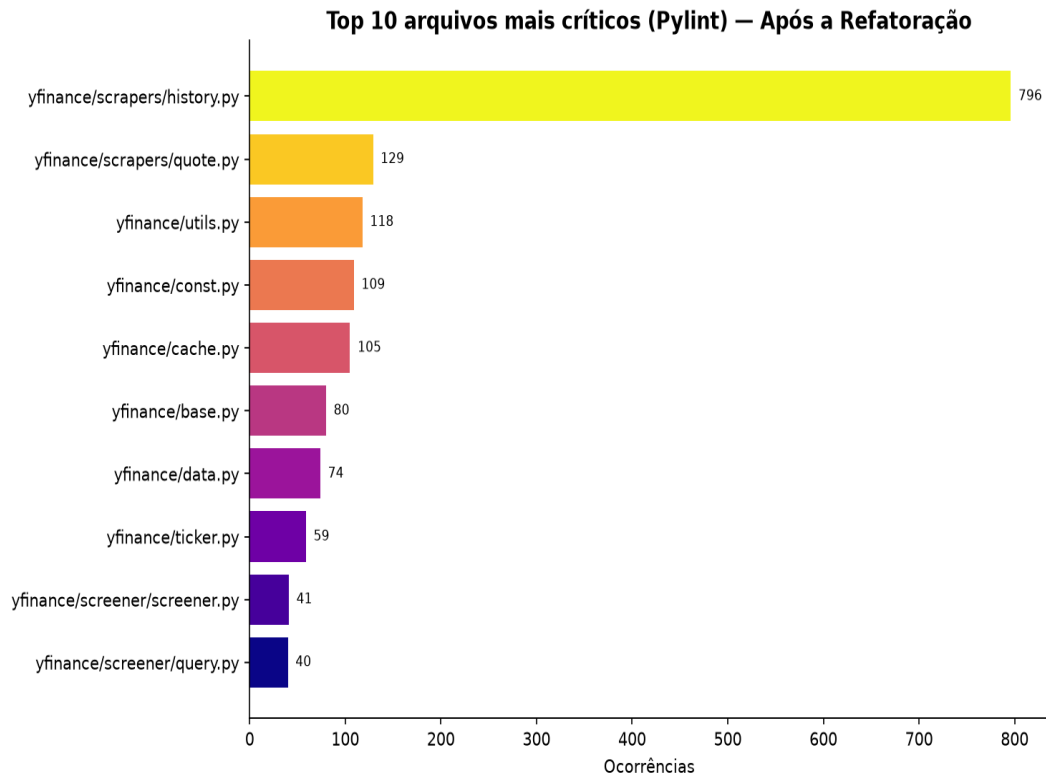
Gráfico gerado a partir de `pylint_distribuicao_categorias_depois.json`:

Distribuição por categoria (Pylint) — Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração

Gráfico gerado a partir de pylint_arquivos_criticos_depois.json:



4.3.4 Score Projeto Após a Refatoração

Score do Pylint: 7.44/10 (Fonte: metrics-after-pylint/pylint_score_depois.txt.)

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Cobertura Total: 78%

(Gráfico comparativo antes/depois apresentado em 2.4.1.)

4.4.2 Testes Passaram vs Testes Falharam Após a Refatoração

Quantidade de testes que passaram: 175

Quantidade de testes que falharam: 6

5. Comparação dos Resultados

A refatoração atingiu integralmente a meta definida para os três code smells-alvo do Pylint. Antes da intervenção, `pylint_refactor_antes.json` registrava 41 ocorrências combinadas (R0915: 12, R0902: 11, R0912: 18); após a refatoração, `pylint_refactor_depois.json` aponta **zero** em todas as três categorias. Isso confirma que as técnicas de Extract Method, agrupamento de estado em estruturas internas e guard clauses eliminaram os smells exigidos sem alterar a API pública do pacote. O score global do Pylint caiu marginalmente (7,86 → 7,44), fenômeno esperado pelo acréscimo de helpers e linhas de código, além de smells de convenção não abordados neste trabalho (ex.: line-too-long, missing-function-docstring).

Quanto ao Radon, houve melhora expressiva na complexidade ciclomática dos pontos críticos. A pior função passou de CC = 245 (`_fix_bad_div_adjust` em `history.py`) para CC = 25 (`camel2title` em `utils.py`), redução de ~90%. O CC máximo de `history.py` caiu de 245 para 15, e o de `multi.py` de 36 para 18. Esse ganho estrutural indica código mais modular e testável. Por outro lado, o volume total de código aumentou (SLOC: 9.170 → 9.988), e o MI de `utils.py` piorou (3,97 → 1,51), efeito colateral da decomposição em muitas funções auxiliares. O arquivo `history.py` permanece com MI = 0,00 por ser o maior módulo do pacote, embora suas funções individuais tenham CC drasticamente menor.

No eixo ambiental e de testes, o CodeCarbon registrou queda de ~54% nas emissões (0,000885 → 0,000409 kg CO₂eq) e no consumo total de energia (0,008997 → 0,004163 kWh), apesar de duração ligeiramente maior (+5,7%). A cobertura de testes subiu de 70,5% para 78%, e o número de testes aprovados aumentou de 164 para 175, com falhas reduzidas de 17 para 6. As falhas remanescentes concentram-se em dependências externas (lxml), chamadas de rede e um teste de repair pré-existente, não indicando regressão funcional direta da refatoração dos smells-alvo.

6. Percepções sobre a Prática

Experiência geral com a atividade: A experiência foi relativamente tranquila, ainda que com alguns percalços e idas e vindas pelo caminho. Boa parte disso veio do próprio funcionamento da LLM: o entendimento dela sobre cada smell dependia bastante do contexto fornecido, então em alguns pontos foi preciso verificar com cuidado o que realmente havia sido feito e refazer trechos que não saíram como esperado, caso mais evidente em `history.py`, que exigiu várias iterações (métodos

truncados, continue vs return, helpers sem @staticmethod). Mesmo assim, isso já era esperado: por já trabalharmos com LLMs no dia a dia, a forma de conduzir a refatoração e de entender o que a ferramenta estava propondo foi bem mais natural do que seria em um primeiro contato com esse tipo de ferramenta.

Code smells: Os três smells-alvo (too-many-statements, too-many-instance-attributes, too-many-branches) são relevantes para a sustentabilidade de um projeto open source porque concentram complexidade em poucos pontos do código, dificultando revisão por terceiros, testes isolados e onboarding de novos contribuidores, efeito visível na redução do CC máximo de history.py de 245 para 15 após a decomposição das funções mais críticas. Entre os smells não tratados neste trabalho, considero importantes line-too-long (442 ocorrências antes da refatoração) e missing-function-docstring (222), por afetarem diretamente a legibilidade e a curva de aprendizado de quem lê o código pela primeira vez, além de protected-access (51) e broad-exception-caught (31), que indicam encapsulamento frágil e tratamento de erro pouco específico, riscos reais em uma biblioteca que faz scraping de APIs externas instáveis. Como principal benefício observado, a refatoração tornou as funções mais críticas testáveis isoladamente e reduziu o consumo de energia medido pelo CodeCarbon em ~54%; como não-benefício, o volume total de código aumentou (SLOC: 9.170 → 9.988) e o score geral do Pylint caiu marginalmente (7,86 → 7,44), já que os helpers novos não vieram com docstrings nem resolveram smells de convenção fora do escopo.

Uso da LLM na refatoração: A LLM entendeu bem os problemas apontados pelo Pylint, as três categorias-alvo foram zeradas (12, 11 e 18 ocorrências, respectivamente) sem alterar a API pública do pacote, e a maioria das sugestões foi diretamente aplicável, em especial Extract Method e guard clauses. Nem toda sugestão inicial foi a melhor: tentativas com dataclass para agrupar atributos voltaram a disparar R0902, exigindo um segundo plano com dicionários simples; substituições em massa no código (self._ → self._cache.) quebraram referências legítimas (self._data, self._symbol) em quote.py; e houve regressão temporária em FastInfo, que acessava self._tkr._quote removido da refatoração de TickerBase. Ainda assim, o uso da LLM acelerou o processo global mais do que o complicou, desde que cada mudança fosse validada com Pylint focalizado e Pytest do módulo antes de seguir para o próximo arquivo. Considero que projetos open source podem se beneficiar bastante dessa abordagem, contanto que mantenham revisão humana e CI como rede de segurança. Além da refatoração em si, a LLM ajudou na contribuição ao gerar os gráficos de métricas a partir dos JSONs brutos do Pylint, extrair e organizar as 41 ocorrências dos smells-alvo a partir de

pylint_raw_antes.json/pylint_raw_depois.json, e estruturar este próprio relatório no formato exigido pelo template da disciplina.

Contribuição com projeto open source: Até o momento, o processo cobriu o fork do repositório, a refatoração local dos três smells-alvo e a validação com Pylint focalizado e Pytest. A submissão do Pull Request para o repositório original (ranaroussi/yfinance) é a próxima etapa, a intenção é abri-los de fato, ainda pendente no momento deste relatório (links a preencher na Seção 7). A principal dificuldade prevista para um projeto do porte do yfinance com alto volume de issues e PRs simultâneos é manter os PRs pequenos e atômicos (um por smell ou por arquivo) para facilitar a revisão dos mantenedores, em vez de um único PR grande cobrindo as 41 ocorrências. O CONTRIBUTING.md do projeto e a suíte de testes existente ajudaram a validar que as mudanças não quebravam o comportamento antes de cogitar a submissão. O que mais facilitaria a contribuição, na minha percepção, seria um guia mais explícito de convenções de estilo (muitos smells de convenção como line-too-long sequer são cobertos por um linter automático no CI do projeto) e exemplos de PRs de refatoração já aceitos, que servem de referência de granularidade esperada pelos mantenedores.

7. Links

- Link do fork: [yfinance fork](#)
- Link do projeto: [yfinance](#)
- Link do PR: [PR Aberto](#)